

GraphRAG Legal Pipeline

Architecture & Design Report

Graph-Based Retrieval-Augmented Generation system for Omani royal decrees, ministerial decisions, and regional legal frameworks.

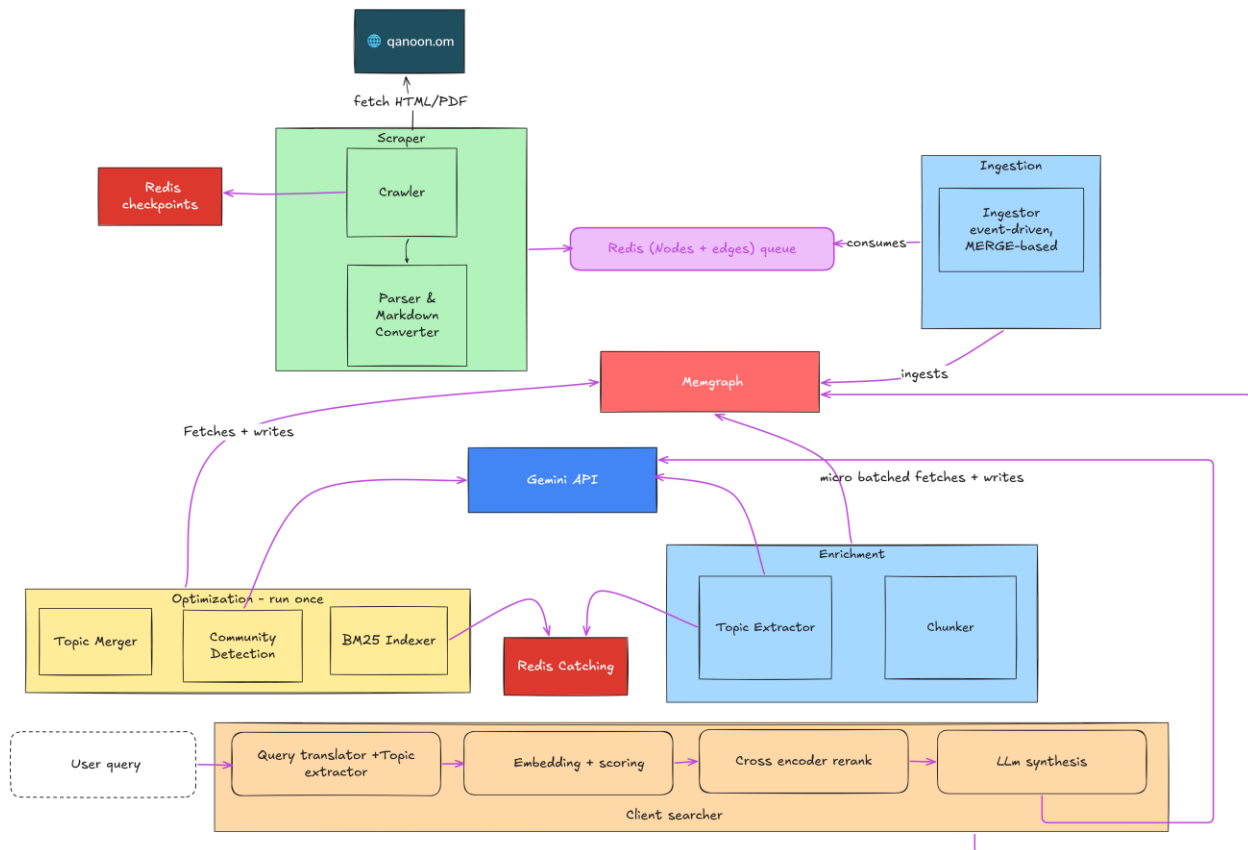
Juin 2026

Work completed by

Frija Farah

1. Overview

This document describes the design of a GraphRAG pipeline that scrapes, structures, and serves Omani legal documents from qanoon.om. The system is built as a set of independent, asynchronous workers connected through two shared stores — **Redis for queues, checkpoints, and other caching techniques**, and **Memgraph for the knowledge graph and its native vector indexes**. Each worker can be started, stopped, and resumed independently, which was the central design constraint: the source site requires polite, throttled crawling over a large and growing document set, so the pipeline has to tolerate being interrupted at any point without losing progress or producing duplicate or corrupted data.



At a high level, the pipeline has six runtime processes:

1. **Crawler** — discovers categories and documents on qanoon.om, converts each page to Markdown, and queues the result.
2. **Ingestor** — drains that queue and writes Document nodes and cross-reference edges into Memgraph.
3. **Topic Extractor** — asks an LLM to label each document with legal topics and links them into the graph.
4. **Chunker** — splits document text into overlapping passages and embeds them for vector search.

1. **Optimization batch** — topic deduplication, Louvain community detection, and BM25 index construction, run once the graph has stabilized.
2. **Search clients** — a simple dense-search CLI and a full multi-stage hybrid search CLI.

 ***The scraper, ingestor, and enrichment components can run in either parallel or sequential mode, as they are decoupled and operate independently of one another.***

The remainder of this document walks through each stage, explains the choices made and the alternatives considered, and is explicit about what is implemented versus what remains as future work.

2. Phase 1 —Crawler & Anti-Bot Evasion

2.1 Discovery strategy

Rather than hardcoding the list of legal categories qanoon.om publishes, the crawler reads them from the site's **WordPress taxonomy sitemap (wp-sitemap-taxonomies-category-1.xml)**.

- **Mapping Strategy:** Each category entry is mapped from its URL slug to an Arabic display name .
- **Pagination Logic:** For each category, listing pages are paginated forward (page/2/, page/3/, ...) until a page returns zero document links, which signals the end of that category.
- **URL Extraction:**
 - Enqueue (url, category_ar) pairs into Redis
 - Save page progress to Redis
 - Mark discovery as done when empty page reached

2.2 Anti-bot evasion

Every outbound request goes through a single fetch() entry point with three layers of defense:

- **Header and identity rotation** — a random User-Agent from a small pool of current desktop browsers, an Accept-Language header weighted toward Arabic, and a Referer pointing back to the site, so requests look like normal browser navigation rather than a script.
- **Pacing — a randomized delay** (1–3s) between requests, plus tenacity-managed exponential backoff (2s up to 60s, 5 attempts) on any HTTP error.

- **Bot-detection fallback** — response bodies are scanned for known blocking signatures (Cloudflare's challenge page, generic CAPTCHA markers, explicit rate-limit messages). If one is found, the plain HTTP client raises a `BlockedError` and the same URL is retried through a headless Playwright browser, which patches `navigator.webdriver` and waits for the page to go network-idle before reading the rendered HTML.

This combination handles the bot defenses a scraper most commonly encounters in practice .

2.3 Checkpointing and resumability

All crawl state lives in Redis under a `qanoon:` prefix, as a small set of well-defined structures:

Key	Type	Purpose
<code>qanoon:visited_urls</code>	SET	Every URL fetched (listing pages and document pages)
<code>qanoon:pending_urls</code>	LIST	Document URLs queued for extraction
<code>qanoon:url_category</code>	HASH	doc URL → category, set during discovery
<code>qanoon:done_doc_ids / failed_doc_ids</code>	SET	Outcome of each document extraction
<code>qanoon:category_progress</code>	HASH	Last fully-processed listing page per category
<code>qanoon:category_state</code>	HASH	DISCOVERING / DISCOVERY_DONE per category

Two properties of this design matter for resumability:

- **Atomic State Transitions:** Every state transition that matters — marking a document done or failed, or enqueueing newly discovered URLs — happens inside a single Redis pipeline, so a crash can never leave a URL simultaneously pending and done, or pending and nowhere.
- **Progress Tracking & Resumption:**
 - A listing page's progress is only recorded after every document URL on it has been enqueued, never mid-page.
 - On restart, the crawler walks backwards to find first fully-done page → restart from there and resumes one page after that.
- **Resilience:** This backward walk is what makes the crawler tolerant to both new documents being added (it naturally re-discovers anything new on the boundary page) and old ones being removed (it doesn't get stuck assuming a page it can no longer find).
- **Failure Handling:** A document extraction failure (network error, parser exception) is recorded in `qanoon:failed_doc_ids` without blocking the rest of the queue.
- **Recovery:** Failed documents can be retried in bulk later with `--retry-failed`, which requeues them and clears their failed/visited state.

3. Phase 2 — Markdown Serialization

Each document can have its body sourced from HTML or from a downloadable PDF.

- **Version-Specific Logic:** The Arabic and English versions do not always come from the same source:
 - **Arabic:** Read from the qanoon.om HTML page when present, falling back to an attached Arabic PDF.
 - **English:** Read from a linked decree.om page, falling back to an attached English PDF.
- **Data Normalization:** Both conversion paths are normalized to the same Markdown shape

3.1 HTML path

- **Conversion Tool:** HTML bodies are converted to markdown, configured to produce ATX-style headings (#, ##, ...) and dash bullets.
- **Tag Sanitization:** The process strips html tags.
- **Heading Normalization:** Because qanoon.om's article markup uses <h3> for individual articles, a small post-pass promotes headings up one level whenever a document has ### but no ##. This ensures documents do not render with a missing top level.
- **Title Formatting:** The document title is always prefixed as an H1.

3.2 PDF path

PDF bodies are extracted with PyMuPDF (fitz), which reads text in block order.

- **Structural Reconstruction:** Since PDFs lack semantic markup, structure is reconstructed heuristically:
 - **Heading Promotion:** A regex matches standard Arabic and English markers (e.g., *الفصل, المادة, Article, Chapter*), promoting matching blocks to H2.
 - **Body Text:** All other content is treated as standard body text.
- **Explicit Failure Modes:** The system detects specific PDF errors to avoid generic exceptions:
 - **ImagePDFError:** Triggers when every page is image-only (scanned with no text layer).
 - **EmptyPDFError:** Triggers when the PDF opens but contains no text blocks.
 - **EncodingErrorPDF:** Triggers when extracted text fails a character-ratio check (e.g., fewer than 15% of the expected script's letters), indicating broken font encoding this problem is redundant in our site's pdfs.

- ❑ **Graceful Degradation:** These failures are logged and treated as "no content available" for the specific language. This ensures that if one language fails, the document can still be ingested successfully with the remaining language populated.
- ❑ **Future work: Integrate OCRS to solve broken front encoding problems and convert the imaged pdfs to text.**

4. Phase 3 — Ingestion

4.1 Ingestion model — event-driven, not request/response

- ❑ **Decoupled Architecture:** The crawler does not write to Memgraph directly. Instead, each parsed document or detected amendment relation is serialized into a JSON envelope (`{type: 'node'|'edge', payload: {...}}`) and pushed onto a single Redis list (`ingestion:docs`).
- ❑ **Ingestor Process:** **In a parallel mode**, separate process pops messages from the queue and MERGEs them into the graph.
- ❑ **Reliability & Retry Logic:**
 - Messages are only removed from the queue after a successful write.
 - On any exception, the message is pushed back onto the queue for a retry.
 - This design ensures a slow or unavailable Memgraph instance does not stall or cause loss of in-flight results; data simply accumulates in Redis.
- ❑ **Independence:** This approach decouples scraping throughput from graph write throughput and allows the ingestor to be restarted independently of the crawler.
- ❑ **Idempotency:** Using MERGE rather than CREATE ensures that ingestion is naturally idempotent. Replaying the same message (e.g., after a crash-induced requeue) converges to the same graph state rather than creating duplicates.

4.2 Node and relationship design

The graph follows the brief's simplified schema deliberately: a single `:Document` node carries both languages as direct properties (`contentAr`, `contentEn`).

Label / type	Key property	properties
<code>:Document</code>	<code>id</code> (unique)	<code>title</code> , <code>date</code> , <code>document_type</code> , <code>issuer</code> , <code>url_ar/en</code> , <code>pdf_url_ar/en</code> , <code>ar_source/en_source</code> , <code>contentAr</code> , <code>contentEn</code> ,

		status
:Topic	name (unique)	embedding (384-dim), community_id (we will see it later)
:Chunk	id (unique)	text, l anguage, doc_id, index, embedding (384-dim)
:Community	id	label (LLM-generated)
(Document)-[:RELATION {type}]->(Document)	—	type='AMENDS', detected from the amendment relation table on category pages
(Document)-[:HAS_TOPIC]- >(Topic)	—	created by the topic extractor
(Document)-[:HAS_CHUNK {language}]->(Chunk)	—	created by the chunker

4.3 Indexes

- **Schema Setup & Idempotency:** The setup process is designed to be run once but is safe to re-run. It establishes essential graph constraints and indexes to maintain data integrity and query performance.
- **Uniqueness Constraints:** To prevent data duplication, the schema enforces uniqueness on:
 - Document.id
 - Topic.name
 - Chunk.id
- **Lookup Indexes:** Performance-oriented indexes are applied to fields frequently used in filter operations, specifically: doc_id
- **Native Vector Indexes:** To support semantic similarity search, the schema implements two native Memgraph vector indexes:
 - :Chunk(embedding)
 - :Topic(embedding)
 - **Dimensionality:** Both indexes are configured to a dimension of 384, matching the embedding model utilized throughout the entire pipeline.

Would you like to understand the performance benefits of using native vector indexes within Memgraph versus external vector storage for this type of document-topic relationship?

5.1 Phase 4 — Enrichement -LLM Topic Extraction

A background worker continuously polls for Document nodes that contain content but lack `:HAS_TOPIC` edges.

□ **LLM Integration:** * The worker calls an LLM (Gemini) with a system prompt to extract 1–7 concise English legal-topic noun phrases.

- **JSON Schema Enforcement:** The output is constrained to a JSON schema, ensuring valid, parseable results and eliminating the need for regex extraction.

□ **Bulk Write Strategy:**

- Results are accumulated in memory for the entire batch.
- Embeds every Topic with paraphrase-multilingual-MiniLM-L12-v2. Writes to Memgraph are performed using two bulk queries: one to upsert Topic nodes and establish Document → Topic relationships.
- This minimizes the Memgraph round-trip cost per document.

□ **Caching & Optimization:** * Topic names are embedded only once.

- A Redis set tracks existing embeddings, ensuring the worker never re-embeds known topics (e.g., 'Taxation' or 'Labor Rights') across different documents or subsequent runs.

=> **Micro-Batching Strategy:** The topic extractor utilizes micro-batching to select a group of Document nodes lacking topics and process them in bulk.

=> **Future-Proofing for LLM Batching:** While the current Gemini API implementation does not support native batching, this architecture is designed to scale seamlessly for future LLM providers that do support batch requests.

=> **Parallel Execution:** This worker is designed to operate in parallel with both the crawler and the ingestor, allowing for independent scaling of scraping, ingestion, and topic extraction processes.

5.1 Phase 4 — Enrichement -Chunking

A second background worker identifies documents lacking `:HAS_CHUNK` edges. These documents are split using **LangChain's** RecursiveCharacterTextSplitter configured for 510< 512 -token segments with a 50-token overlap).

□ **Embedding Model:** Every chunk is embedded using paraphrase-multilingual-MiniLM-L12-v2. This free, self-hosted sentence-transformer was selected because it supports 50+ languages (including Arabic), enabling a unified embedding space for both languages .

□ **Bulk Ingestion:** The resulting Chunk nodes are bulk-inserted using a single query, with each node linked to its parent Document via a relationship that includes a language property.

□ **Current Language Strategy:** The system currently chunks whichever language is present, defaulting to Arabic if both are available. This was a scope-simplification for initial deployment.

=> **Micro-Batching Strategy:** The chunker uses a micro batching strategy to not cause a bottleneck writing in the whole memgraph nodes on the same time so it locks the whole db for too long.

6.1 Phase 5 — Optimization

6.1 Topic deduplication

To address LLM-generated topic variability (e.g., 'Labor Policies' vs. 'Labor Regulations'), the system uses a deduplication script to merge synonymous concepts.

□ **Similarity Graph Approach:**

- The script builds a similarity graph where edges exist between topics with cosine similarity > 0.88 .
- It identifies **connected components** rather than simple pairs. This allows for transitive merging (A~B and B~C merges A, B, and C into one cluster and they are all replaced by A), capturing synonyms that may not directly meet the threshold.

□ **Canonical Selection:** Within each cluster, the topic node with the most :HAS_TOPIC edges is retained as the canonical representative. All document links are redirected to this node, and duplicate nodes are deleted.

□ **Precision-Focused Tuning:** The 0.88 threshold is intentionally conservative to prioritize precision over recall.

- Near-duplicates typically score > 0.85 .
- Distinct but related concepts (e.g., 'Taxation' vs. 'Customs Duties') typically score lower.
- This minimizes false merges, which are harder to reverse than missed merges.

6.2 Community detection

The system uses Memgraph's MAGE `community_detection.get_subgraph` procedure to identify clusters within the graph.

□ **Subgraph Projection:** Analysis is limited strictly to Document and Topic nodes connected via :HAS_TOPIC edges. Chunk nodes and AMENDS edges are excluded to prevent structural noise from diluting topical clusters.

□ **LLM-Based Labeling:** Once the Louvain algorithm assigns a `community_id` to nodes, a worker sends the associated topic names and document titles to an LLM. The model generates a concise, human-readable noun phrase (max 8 words) describing the legal sub-field.

□ **Semantic Indexing:** The resulting :Community node is persisted and linked to its members via :BELONGS_TO. This creates an emergent, human-readable index of legal domains discovered directly from the data, rather than relying on predefined categories.

7.1 Phase 6 — Hybrid search with multi-stage reranking

This is the most involved part of the system. A query is answered in three stages:

Stage 1 — Candidate generation (top-K=50, default)

Three search strategies run in parallel against a thread pool, as they query different backends (BM25, Chunk Vector Index, Topic Vector Index) without interdependencies.

□ Search Strategies:

- **Sparse (BM25):** Uses separate BM25Okapi indexes for English and Arabic (pickled in Redis). Arabic tokenization filters for Arabic-script; English uses lowercase and word-boundary splits. The document score is the maximum of its EN and AR matches.
- **Dense (Chunks):** The query vector is generated and searched directly against the Chunk index using `vector_search.search()`.
- **Dense (Topics):** The query is sent to an LLM to extract implied legal topics. These topics are embedded and averaged into a single vector. Documents linked to these topics inherit the resulting similarity score.

□ **Bilingual Query Handling:** To ensure language-agnostic retrieval, every query is embedded in both English and Arabic. The resulting vectors are averaged into a single query vector before the dense searches are executed.

□ **Weighted Scoring:** Results from all sources are independently normalized to $[0, 1]$ and aggregated using a weighted sum:

- **\$0.6\$ (Dense Chunks):** Primary signal for semantic relevance.
- **\$0.2\$ (Dense Topics):** Supporting semantic context.
- **\$0.2\$ (BM25):** Supporting keyword-based evidence.

Stage 2 — Topological context expansion

For every candidate chunk surviving Stage 1, its parent Document's metadata (title, issuer, date, document_type) and linked Topic names are fetched and prepended to the chunk's text before reranking, so the reranker sees not just the passage but which law it's from and what it's broadly about. This is done as a single batched Memgraph query across all K candidates rather than one query per chunk, and the per-document metadata is cached in Redis with a one-hour TTL — repeated or related queries that surface chunks from the same handful of frequently-cited documents (a common pattern in legal search) skip the graph round-trip entirely after the first hit.

Stage 3 — Cross-encoder reranking and synthesis

Reranking Phase: Each (query, expanded_chunk) pair is evaluated by a BAAI/bge-reranker-base cross-encoder. Unlike the Stage 1 bi-encoders, this model processes the query and candidate passage in a single forward pass, providing significantly higher precision.

- **Efficiency Constraint:** Because cross-encoders are computationally intensive and not precomputable, they are only applied to the top-K results from Stage 1 rather than the full corpus.
- **Synthesis:** The top final_n (default 5) reranked passages are provided to the LLM to synthesize a grounded final answer, including citations for document titles and provision numbers.
- **Latency & Cost Optimization:**
 - **Concurrency:** Stage 1 searches (BM25, Dense Chunks, Dense Topics) execute in parallel.
 - **Precomputation:** BM25 indexes are built once and loaded into memory at startup; query-time cost is limited to a simple lookup.
 - **Model Persistence:** The cross-encoder is loaded into memory once at startup, not per request.
 - **Batching:** Stage 2 graph traversals are **batched and cached** to reduce round-trips to approximately one per request.
- **Future Development Roadmap:**
 - **Quantization:** Reranker model optimization is pending.
 - **Benchmarking:** Formal measurement of latency, precision, recall, and Mean Reciprocal Rank (MRR) against a labeled query set is required to transition from the current implementation to a fully evaluated production system.

8.Design Trade-offs

Why memgraph (in-memory graph database) and not Neo4j (disk-based graph database)?

1. Performance & Real-Time Analytics

- **In-Memory Architecture:**
Memgraph stores and processes data entirely in memory, which drastically reduces query latency compared to Neo4j's disk-based persistence model.
 - Ideal for **real-time searches** .
 - while ensuring [durability](#) through **periodic snapshots** and **write-ahead logging (WALs)** in transactional mode.
- **High Throughput:**
Handles millions of edges and nodes with sub-millisecond traversal times, making it better suited for **low-latency, high-frequency queries**.
- **Best for dynamic graphs** where data changes frequently (in case we will scrape continuously or the scraping takes days) .

2. Cost Efficiency

Licensing Model: Memgraph offers a more cost-effective licensing structure compared to Neo4j Enterprise, which can become expensive at scale

3. Scalability

- Best for **small-to-medium datasets** that fit in memory , so the choice fits our site's qanoon.com size.
- If the number of documents grows exponentially or we add other ingestion sources , we can use **Memgraph** for real-time ingestion (streaming or microbatching) and fast retrieval.
and Periodically sync to **Neo4j** for the topic richer batching analytics (the topic merger for example).

4. Integration

- **MAGE (Memgraph Advanced Graph Extensions):** Built-in algorithms such as Louvain for graph analytics, embeddings, and ML integration

Why redis for checkpoints, topics catching ?

Redis is ideal for progress checkpointing because it combines high-speed in-memory access with configurable persistence, enabling agents to save intermediate state and resume tasks reliably after crashes or interruptions.

High-Speed In-Memory Storage

This is crucial for long-running agents or workflows that need to frequently update their progress without introducing latency that would slow down task execution

Persistence Options for Durability

Redis provides multiple persistence mechanisms that make checkpointing reliable:

- **RDB (Redis Database) snapshots:** Periodic point-in-time snapshots of the dataset allow recovery from a specific moment, which is useful for disaster recovery

Atomicity

Redis transactions allow grouping multiple commands into a single atomic operation
For example : If the program breaks after popping a pending url from the pending queue and before pushing it into the ingestion queue the event won't be list since the two transactions are executed atomically)

Why the crawling worker , ingestion and chunker are working in a parellel mode?

To gain time and not wait for the scraper to scrape the hole site which can take sometimes many and many hours.

But our work can be run in a sequential mode since the workers are independent.

Why Ingestion queue ?

To make the crawler independent from the ingestor, the scraped document should be saved somewhere , it's better to not make the saving depend on the ingestion and create a decoupling.

Second, we use micro-batching to optimize performance. This approach provides flexibility; if the scraper's output speed is high, or if the ingestor starts after the scraper has already accumulated a backlog, we can ingest micro-batches.

Why translating the query ?

The BM25 vector is language dependent , since the two languages are not always present in our nodes , we need two versions so that the vector becomes meaningful.

Why getting the query's topics?

Comparing a raw query vector against a topic vector can introduce noise; it is more accurate to compare the query's extracted topics directly against the topic index..

Why the chunk size is set to 510 tokens?

Chunk size is set to 510 tokens — 2 tokens below the model's 512-token context limit — to safely accommodate the 2 special tokens ([CLS]/[SEP]) added at inference time.

9. Known Limitations & Next Steps

In rough priority order:

3. REPEALS relationships are not yet detected — only AMENDS is implemented, via the relation table present on amended-category pages. Adding REPEALS would need a separate detection signal .
4. The cross-encoder is not quantized; running it in int8 (e.g. via ONNX Runtime or bitsandbytes) is a reasonable next optimization once a latency baseline exists to measure against.
5. LLM calls in the topic extractor and community summarizer are made one document/community at a time since the batching is not supported by the gemini free api key. At larger scale, batching multiple documents into a single structured-output prompt would reduce both wall-clock time and per-call overhead.
6. The BM25 index is a single pickled object loaded fully into each search process's memory. This is fine at the current corpus size but won't scale indefinitely; a dedicated sparse-search service (e.g. OpenSearch) would be the natural replacement if the corpus grows substantially.